

Table 1: Important libraries for AI applications development

TensorFlow	Keras	PyTorch	Scikit-Learn
NumPy	Matplotlib	Pandas	Seaborn
OpenCV	NLTK	Pillow	SpaCy
Gensim	Transformers	HuggingFace	FastAI
XGBoost	CatBoost	LightGBM	Optuna
PyCaret	ONNX	MLflow	TFLite
OpenAI Gym	Stable Baselines	Ray RLlib	PettingZoo
Dask	RAPIDS	CuPy	PyTorch Geometric
Plotly	Dash	Bokeh	Holoviews
AllenNLP	YOLO	Detectron2	DeepFace
Chainer	Horovod	Gluon	Ludwig
Eli5	LIME	SHAP	Yellowbrick
CatBoost	XGBoost	LightGBM	H2O.ai
PyTorch Ignite	SentencePiece	FastText	Flair
AutoKeras	KNN	TFX	SVM
Prophet	ARIMA	StatsModels	Facebook Kats
OpenMMLab	MMSegmentation	MMDetection	MMPose
DeepChem	PyTorch Geometric	RDKit	DGL
Detecto	imgaug	Albumentations	Kornia
TorchAudio	Librosa	torchaudio	SoundFile
Turi Create	ONNX Runtime	CoreML	TensorRT

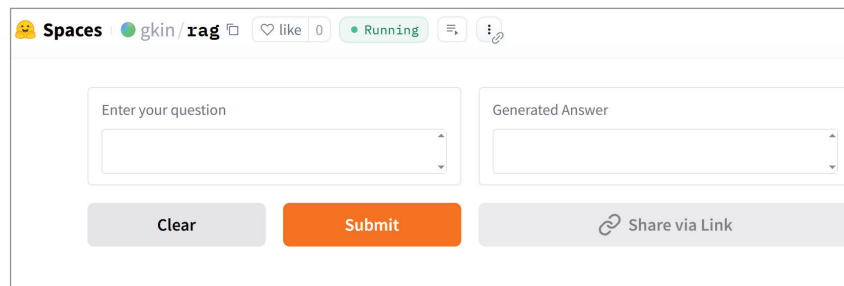


Figure 4: Deployed application on Hugging Face Spaces

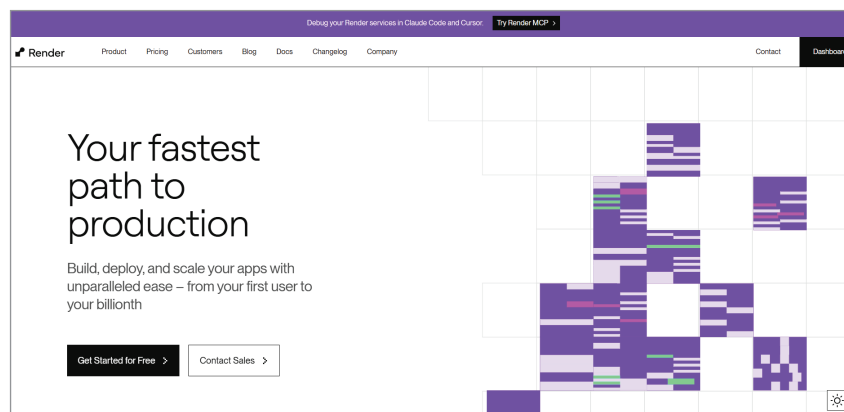


Figure 5: Render, a key cloud platform for deployment of AI applications

a web-based application.

app.py

```
import numpy as np
import faiss
from transformers import
T5ForConditionalGeneration, T5Tokenizer
from sentence_transformers import
SentenceTransformer
import gradio as grgui

# -----
# Step 1: Load models
# -----
myretriever = SentenceTransformer('all-
MiniLM-L6-v2') # Small, fast embedding
model
mygenerator =
T5ForConditionalGeneration.from_
pretrained('t5-small')
mytokenizer = T5tokenizer.from_
pretrained('t5-small')

# -----
# Step 2: Knowledge base for Training
and Adding own documents
# -----
documents = [
    "Red Fort is in Delhi",
    "Great Wall of China is one of the
longest walls in the world.",
    "Python is a prominent programming",
    "Boiling point of water 100 degrees
Celsius in standard conditions"
]
```

```
# Converting the documents to embeddings
mydoc_embeddings = myretriever.
encode(documents, convert_to_numpy=True)

# Create FAISS index
mydimension = doc_embeddings.shape[1]
myindex = faiss.
IndexFlatL2(mydimension)
myindex.add(mydoc_embeddings)

# -----
# Step 3: RAG query function
# -----
def myrag_query(question):
```